



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)

НАПРАВЛЕНИЕ ПОДГОТОВКИ 09.04.01 Информатика и вычислительная техника

МАГИСТЕРСКАЯ ПРОГРАММА 09.04.01/07 Интеллектуальные системы анализа,
обработки и интерпретации больших данных

О Т Ч Е Т

по лабораторной работе № 2

Название: генератор числовых данных

Дисциплина: информационная безопасность в системах поддержки принятия
решений

Студент

ИУ6-42М

(Группа)

(Подпись, дата)

Д.А.Залимханов

(И.О. Фамилия)

Преподаватель

(Подпись, дата)

Д.А. Миков

(И.О. Фамилия)

Москва, 2025

Цель: формирование числовых данных для проверки работоспособности распределенной системы.

Задание: переписать программу генерацию данных, написанную на языке pascal.

В качестве основы для прикладной модели цепочки принята модель отдельного элемента. Математическое его описание для уровня $h = 2$ с использованием задачи линейного программирования имеет вид:

$$\begin{aligned}PN &\leq P \leq PV, \\ BN &\leq DP \leq BV, \\ G &= FP \rightarrow \max,\end{aligned}$$

где P — вектор искомого плана; PN , PV — векторы нижнего и верхнего ограничений на план; D — матрица норм расходов ресурсов; F — вектор прибыли от единицы плановой продукции; BN , BV — векторы нижней и верхней границ ресурсов. Выходом является вектор P , входом — вектор $B = (BN, BV)$. Это прямая задача линейного программирования: по PV , D , BV , F найти P

При моделировании технологической линии необходимо решать обратную задачу: по P , D , DV , F найти $BV=B$ с учетом условий:

$$(0, BV)_k = (0, PV)_{k-1}, BV_k = PV_{k-1}, P_k = B_{k-1} = D_{k-1} \cdot P_{k-1},$$

где k - номер структурного элемента (последний элемент $k = 1$, первый - $k = K$).

Код переписанной программы:

```
import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,
                              QLabel,
                              QLineEdit, QPushButton, QFrame, QProgressBar, QTextEdit, QGroupBox,
                              QGridLayout, QDialog, QDialogButtonBox, QMessageBox, QSpinBox)
import numpy as np
import random
import os
class OptimizationGenerator(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Генератор задач оптимизации")
        self.resize(800, 600)

        # Константы
        self.MMAX = 10
        self.NMAX = 10
        self.TMAX = 10
        self.IMAX = 50
```

```

# Переменные для хранения данных
self.Iter = 1 # Число временных интервалов
self.Iter = 1 # Число решаемых задач

# Массивы для данных
self.M = np.zeros(self.IMAX, dtype=int)
self.N = np.zeros(self.IMAX, dtype=int)
self.A = np.zeros((self.TMAX, self.MMAX, self.NMAX), dtype=float)
self.C = np.zeros(self.NMAX, dtype=float)
self.DN = np.zeros(self.NMAX, dtype=float)
self.DV = np.zeros(self.NMAX, dtype=float)
self.DEL = np.zeros(self.NMAX, dtype=float)
self.BN = np.zeros(self.MMAX, dtype=float)
self.BV = np.zeros(self.MMAX, dtype=float)
self.Y = np.zeros(self.MMAX, dtype=float)
self.X = np.zeros(self.NMAX, dtype=float)
self.AX = np.zeros(self.MMAX, dtype=float)
self.DB = np.zeros(self.MMAX, dtype=float)
self.CX = 0.0

# Создание центрального виджета
self.central_widget = QWidget()
self.setCentralWidget(self.central_widget)
self.main_layout = QVBoxLayout(self.central_widget)

# Создание основного интерфейса
self.create_widgets()

def create_widgets(self):
    # Фрейм для ввода параметров
    param_group = QGroupBox("Параметры генерации")
    param_layout = QGridLayout()
    param_group.setLayout(param_layout)

    # Ввод числа временных интервалов
    param_layout.addWidget(QLabel("Число временных интервалов:"), 0, 0)
    self.inter_spin = QSpinBox()
    self.inter_spin.setMinimum(1)
    self.inter_spin.setValue(self.Iter)
    self.inter_spin.valueChanged.connect(self.update_inter)
    param_layout.addWidget(self.inter_spin, 0, 1)

    # Ввод числа решаемых задач
    param_layout.addWidget(QLabel("Число решаемых задач:"), 1, 0)
    self.iter_spin = QSpinBox()
    self.iter_spin.setMinimum(1)
    self.iter_spin.setValue(self.Iter)
    self.iter_spin.valueChanged.connect(self.update_iter)
    param_layout.addWidget(self.iter_spin, 1, 1)

    self.main_layout.addWidget(param_group)

    # Кнопка запуска генерации
    self.gen_button = QPushButton("Начать генерацию")
    self.gen_button.clicked.connect(self.start_generation)
    self.main_layout.addWidget(self.gen_button)

```

```

# Фрейм для прогресса
progress_group = QGroupBox("Прогресс")
progress_layout = QVBoxLayout()
progress_group.setLayout(progress_layout)

# Индикатор прогресса
self.progress = QProgressBar()
progress_layout.addWidget(self.progress)

self.main_layout.addWidget(progress_group)

# Кнопка очистки лога
self.clear_button = QPushButton("Очистить лог")
self.clear_button.clicked.connect(self.clear_log)
self.main_layout.addWidget(self.clear_button)

# Текстовое поле для вывода информации
self.log_area = QTextEdit()
self.log_area.setReadOnly(True)
self.main_layout.addWidget(self.log_area)

def update_inter(self, value):
    """Обновляет значение числа временных интервалов"""
    self.Inter = value

def update_iter(self, value):
    """Обновляет значение числа решаемых задач"""
    self.Iter = value

def log_message(self, message):
    """Добавляет сообщение в лог с прокруткой в конец"""
    self.log_area.append(message)
    # Прокрутка в конец
    cursor = self.log_area.textCursor()
    cursor.movePosition(cursor.End)
    self.log_area.setTextCursor(cursor)
    QApplication.processEvents() # Обновляем UI

def clear_log(self):
    """Очищает текстовое поле лога"""
    self.log_area.clear()

def rand1(self, rfrom, rto):
    """Аналог функции Rand1 из Pascal-программы"""
    d = random.random()
    d = rfrom + d * (rto - rfrom)
    i = round(d * 10)
    return i / 10

def max_x(self, val, size):
    """Аналог функции MaxX из Pascal-программы"""
    if size > 0:
        return max(val[:size])
    return 0

def min_x(self, val, size):
    """Аналог функции MinX из Pascal-программы"""
    if size > 0:

```

```

        return min(val[:size])
    return 0

def input_dialog(self, title, message):
    """Создает диалоговое окно для ввода значения"""
    dialog = QDialog(self)
    dialog.setWindowTitle(title)
    dialog.resize(300, 150)

    layout = QVBoxLayout(dialog)
    layout.addWidget(QLabel(message))

    line_edit = QLineEdit()
    layout.addWidget(line_edit)

    button_box = QDialogButtonBox(QDialogButtonBox.Ok)
    button_box.accepted.connect(dialog.accept)
    layout.addWidget(button_box)

    result = None
    if dialog.exec_() == QDialog.Accepted:
        try:
            result = float(line_edit.text())
        except ValueError:
            QMessageBox.warning(self, "Ошибка", "Введено некорректное значение")
            return self.input_dialog(title, message)

    return result

def matrix_input_dialog(self, title, rows, cols):
    """Создает диалоговое окно для ввода матрицы"""
    dialog = QDialog(self)
    dialog.setWindowTitle(title)
    dialog.resize(max(300, cols * 100), max(200, rows * 50))

    layout = QVBoxLayout(dialog)

    grid = QGridLayout()
    entries = []

    for i in range(rows):
        row_entries = []
        for j in range(cols):
            grid.addWidget(QLabel(f"A[{i+1},{j+1}]:"), i, j*2)
            entry = QLineEdit()
            entry.setFixedWidth(80)
            grid.addWidget(entry, i, j*2+1)
            row_entries.append(entry)
        entries.append(row_entries)

    layout.addLayout(grid)

    button_box = QDialogButtonBox(QDialogButtonBox.Ok)
    button_box.accepted.connect(dialog.accept)
    layout.addWidget(button_box)

    if dialog.exec_() == QDialog.Accepted:
        # Преобразуем введенные значения в матрицу

```

```

matrix = np.zeros((rows, cols))
try:
    for i in range(rows):
        for j in range(cols):
            matrix[i, j] = float(entries[i][j].text())
    return matrix
except ValueError:
    QMessageBox.warning(self, "Ошибка", "Введено некорректное значение")
    return self.matrix_input_dialog(title, rows, cols)

return None

def vector_input_dialog(self, title, message, size):
    """Создает диалоговое окно для ввода вектора"""
    dialog = QDialog(self)
    dialog.setWindowTitle(title)
    dialog.resize(300, max(200, size * 40))

    layout = QVBoxLayout(dialog)
    layout.addWidget(QLabel(message))

    grid = QGridLayout()
    entries = []

    for i in range(size):
        grid.addWidget(QLabel(f"X[{i+1}]:"), i, 0)
        entry = QLineEdit()
        grid.addWidget(entry, i, 1)
        entries.append(entry)

    layout.addLayout(grid)

    button_box = QDialogButtonBox(QDialogButtonBox.Ok)
    button_box.accepted.connect(dialog.accept)
    layout.addWidget(button_box)

    if dialog.exec_() == QDialog.Accepted:
        # Преобразуем введенные значения в вектор
        vector = np.zeros(size)
        try:
            for i in range(size):
                vector[i] = float(entries[i].text())
            return vector
        except ValueError:
            QMessageBox.warning(self, "Ошибка", "Введено некорректное значение")
            return self.vector_input_dialog(title, message, size)

    return None

def start_generation(self):
    """Основной метод для генерации задач, аналог основной программы Pascal"""
    try:
        inter = self.Inter
        iter_count = self.Iter

        if inter <= 0 or iter_count <= 0:
            QMessageBox.critical(self, "Ошибка", "Значения должны быть положительными")
            return

```

```

# Очищаем лог перед началом
self.clear_log()

# Настраиваем прогресс-бар
total_steps = inter * iter_count
self.progress.setMaximum(total_steps)
self.progress.setValue(0)

# Основной цикл генерации (аналог вложенных циклов в Pascal)
for l in range(1, inter + 1):
    self.log_message(f"Временной интервал #{l}")

    for k in range(1, iter_count + 1):
        self.log_message(f"Задача #{k}")

        # Создаем файл для записи результатов
        filename = f"out_{l}_{k}.txt"
        with open(filename, "w") as fo:
            if l == 1:
                # Задаем размерность задачи для первого интервала
                self.log_message(f"Введите размерность задачи #{k}")

                if k == 1:
                    n_k = int(self.input_dialog("Ввод размерности", "N="))
                    self.N[k-1] = n_k
                else:
                    self.N[k-1] = self.M[k-2]
                    self.log_message(f"N={self.N[k-1]}")

                m_k = int(self.input_dialog("Ввод размерности", "M="))
                self.M[k-1] = m_k

                fo.write(f"{self.N[k-1]:2}<==N\n")
                fo.write(f"{self.M[k-1]:2}<==M\n")

                # Проверка на допустимость размерностей
                if self.N[k-1] > self.NMAX or self.M[k-1] > self.MMAX:
                    self.log_message("Too big dimensions!")
                    QMessageBox.critical(self, "Ошибка", "Слишком большие размерности!")
                    return

            if k == 1:
                # Для первой задачи запрашиваем план X
                self.log_message("Введите план")
                self.X[:self.N[k-1]] = self.vector_input_dialog(
                    "Ввод плана X", "Введите компоненты вектора X:", self.N[k-1])

            if k > 1 and l > 1:
                # Для последующих задач запрашиваем deltaB
                self.log_message("Введите deltaB")
                for i in range(self.M[k-1]):
                    self.log_message(f"Введите dB для задачи #{k}")
                    self.DB[i] = self.input_dialog("Ввод deltaB", f"dB[{i+1}]=")

        # Вычисление DN, DV, DEL
        for j in range(self.N[k-1]):
            if k == 1:

```

```

min_x_val = self.min_x(self.X[:self.N[k-1]], self.N[k-1])
max_x_val = self.max_x(self.X[:self.N[k-1]], self.N[k-1])
threshold1 = min_x_val + 0.35 * (max_x_val - min_x_val)
threshold2 = min_x_val + 0.65 * (max_x_val - min_x_val)

if self.X[j] < threshold1:
    self.DN[j] = self.X[j]
    self.DV[j] = max_x_val
    self.DEL[j] = self.rand1(
        self.DN[j] + 0.25 * (self.DV[j] - self.DN[j]),
        self.DN[j] + 0.45 * (self.DV[j] - self.DN[j])
    )
elif self.X[j] <= threshold2:
    self.DN[j] = min_x_val
    self.DV[j] = max_x_val
    self.DEL[j] = (max_x_val - min_x_val) / 2
else:
    self.DN[j] = min_x_val
    self.DV[j] = self.X[j]
    self.DEL[j] = self.rand1(
        self.DN[j] + 0.55 * (self.DV[j] - self.DN[j]),
        self.DN[j] + 0.75 * (self.DV[j] - self.DN[j])
    )
else:
    self.DN[j] = self.BN[j]
    self.DV[j] = self.BV[j]
    self.X[j] = self.AX[j] + self.DB[j]

threshold1 = self.DN[j] + 0.35 * (self.DV[j] - self.DN[j])
threshold2 = self.DN[j] + 0.65 * (self.DV[j] - self.DN[j])

if self.X[j] < threshold1:
    self.DEL[j] = self.rand1(
        self.DN[j] + 0.25 * (self.DV[j] - self.DN[j]),
        self.DN[j] + 0.45 * (self.DV[j] - self.DN[j])
    )
elif self.X[j] <= threshold2:
    self.DEL[j] = (self.DN[j] + self.DV[j]) / 2
else:
    self.DEL[j] = self.rand1(
        self.DN[j] + 0.55 * (self.DV[j] - self.DN[j]),
        self.DN[j] + 0.75 * (self.DV[j] - self.DN[j])
    )

# Ввод или вычисление матрицы A и вектора AX
if l == 1:
    self.log_message("Введите матрицу A")
    matrix_a = self.matrix_input_dialog(
        "Ввод матрицы A", self.M[k-1], self.N[k-1])

# Копируем матрицу A в трехмерный массив и вычисляем AX
for i in range(self.M[k-1]):
    self.AX[i] = 0
    for j in range(self.N[k-1]):
        self.A[k-1, i, j] = matrix_a[i, j]
        self.AX[i] += self.A[k-1, i, j] * self.X[j]
else:
    # Для последующих интервалов вычисляем AX

```

```

for i in range(self.M[k-1]):
    self.AX[i] = 0
    for j in range(self.N[k-1]):
        self.AX[i] += self.A[k-1, i, j] * self.X[j]

# Вычисление BN, BV, Y
for i in range(self.M[k-1]):
    min_ax_val = self.min_x(self.AX[:self.M[k-1]], self.M[k-1])
    max_ax_val = self.max_x(self.AX[:self.M[k-1]], self.M[k-1])
    threshold1 = min_ax_val + 0.35 * (max_ax_val - min_ax_val)
    threshold2 = min_ax_val + 0.65 * (max_ax_val - min_ax_val)

    if self.AX[i] < threshold1:
        self.BN[i] = self.AX[i]
        self.BV[i] = max_ax_val
        self.Y[i] = self.rand1(
            self.BN[i] + 0.25 * (self.BV[i] - self.BN[i]),
            self.BN[i] + 0.45 * (self.BV[i] - self.BN[i])
        )
    elif self.AX[i] <= threshold2:
        self.BN[i] = min_ax_val
        self.BV[i] = max_ax_val
        self.Y[i] = (max_ax_val - min_ax_val) / 2
    else:
        self.BN[i] = min_ax_val
        self.BV[i] = self.AX[i]
        self.Y[i] = self.rand1(
            self.BN[i] + 0.55 * (self.BV[i] - self.BN[i]),
            self.BN[i] + 0.75 * (self.BV[i] - self.BN[i])
        )

# Вычисление C и CX
self.CX = 0
for j in range(self.N[k-1]):
    self.C[j] = self.DEL[j]
    for i in range(self.M[k-1]):
        self.C[j] += self.A[k-1, i, j] * self.Y[i]
    self.CX += self.C[j] * self.X[j]

# Запись результатов в файл
fo.write(f"C, CX={self.CX:6.2f}\n")
fo.write(" ".join([f"{self.C[j]:6.2f}" for j in range(self.N[k-1])]) + "\n")

fo.write("DN\n")
fo.write(" ".join([f"{self.DN[j]:6.2f}" for j in range(self.N[k-1])]) + "\n")

fo.write("DV\n")
fo.write(" ".join([f"{self.DV[j]:6.2f}" for j in range(self.N[k-1])]) + "\n")

fo.write("BN\n")
fo.write(" ".join([f"{self.BN[i]:6.2f}" for i in range(self.M[k-1])]) + "\n")

fo.write("BV\n")
fo.write(" ".join([f"{self.BV[i]:6.2f}" for i in range(self.M[k-1])]) + "\n")

fo.write("A\n")
for i in range(self.M[k-1]):
    fo.write(" ".join([f"{self.A[k-1, i, j]:6.2f}" for j in range(self.N[k-1])]) + "\n")

```

```

fo.write("X\n")
fo.write(" ".join([f"{self.DV[j]:6.2f}" for j in range(self.N[k-1])]) + "\n")

fo.write("Xopt\n")
fo.write(" ".join([f"{self.X[j]:6.2f}" for j in range(self.N[k-1])]) + "\n")

fo.write("B\n")
fo.write(" ".join([f"{self.AX[i]:6.2f}" for i in range(self.M[k-1])]) + "\n")

self.log_message(f"Результаты сохранены в файл {filename}")

# Обновляем прогресс-бар
self.progress.setValue(self.progress.value() + 1)
QApplication.processEvents()

QMessageBox.information(self, "Готово", "Генерация завершена успешно!")

except Exception as e:
    QMessageBox.critical(self, "Ошибка", f"Произошла ошибка: {str(e)}")
    self.log_message(f"ОШИБКА: {str(e)}")

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = OptimizationGenerator()
    window.show()
    sys.exit(app.exec_())

```

На рисунках 1 – 5 показана работа программы. На рисунке 6 показан результат работы программы.

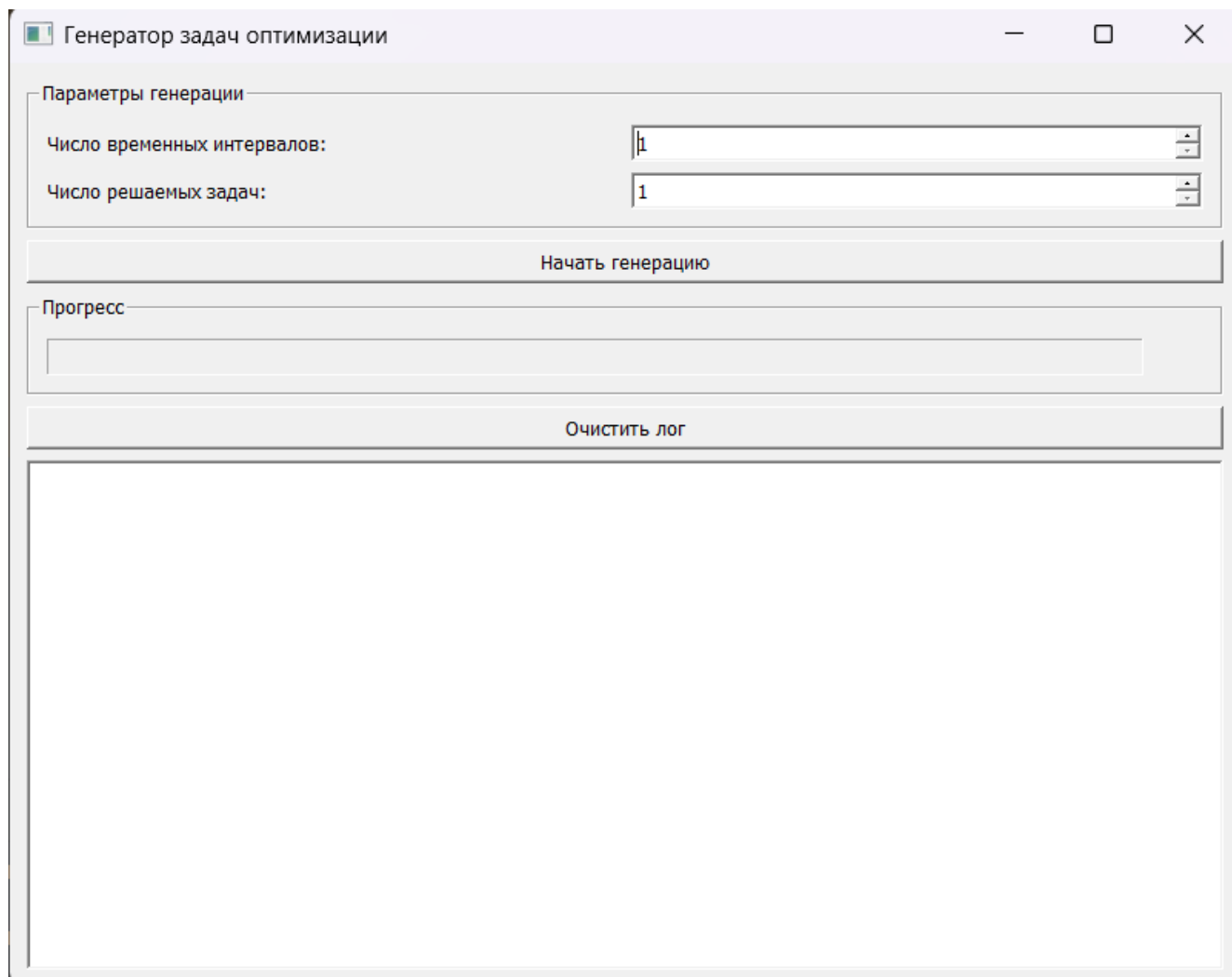


Рисунок 1 – Начальный экран

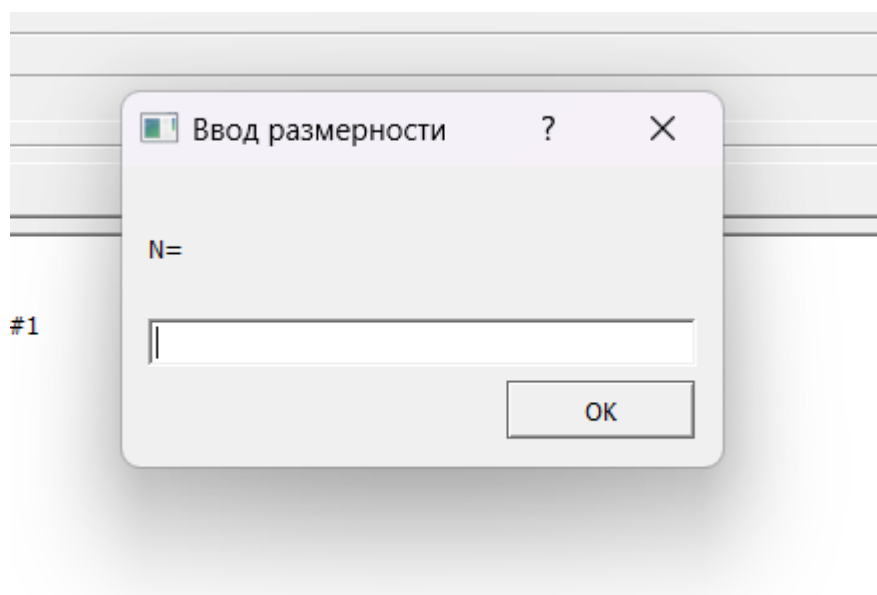


Рисунок 2 – Ввод размерности задачи

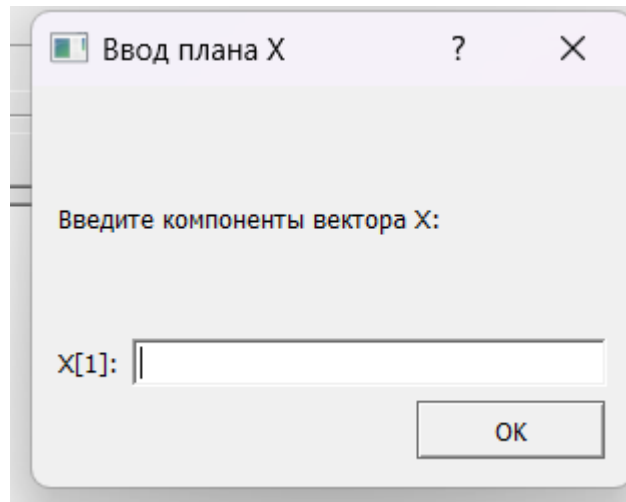


Рисунок 3 – Ввод плана

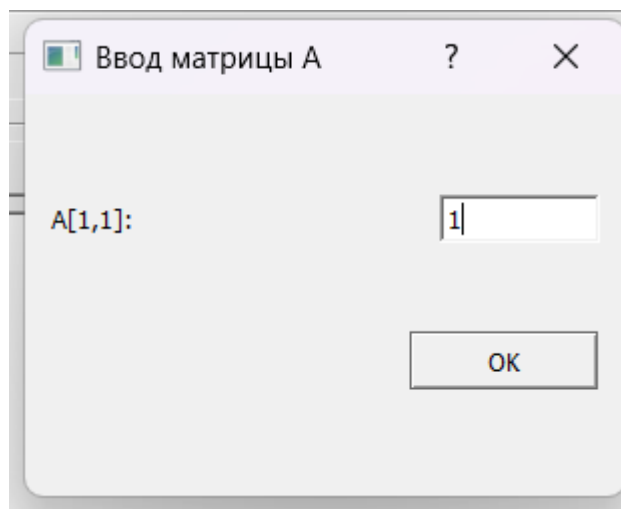


Рисунок 4 – Ввод матрицы

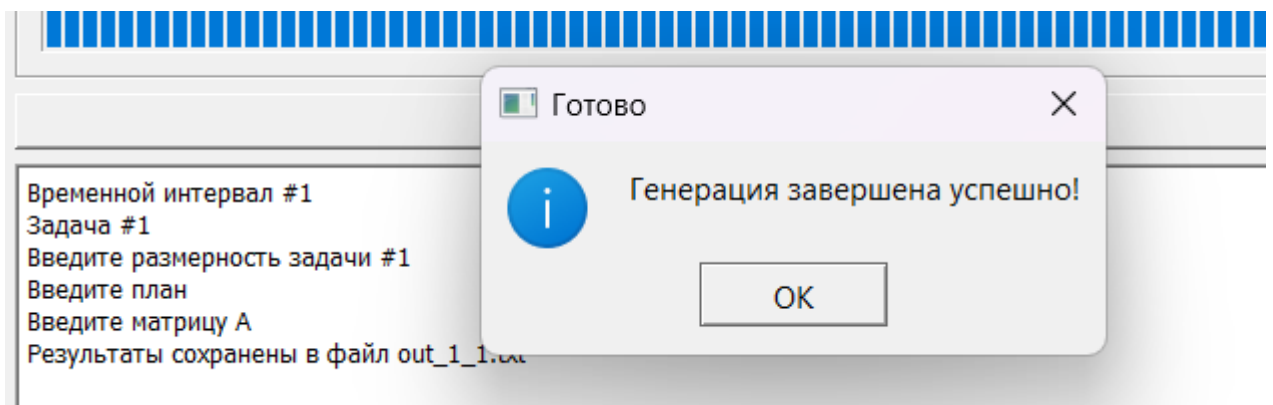


Рисунок 5 – Сообщение о результате работы

```
1<=N
1<=M
C, CX= 0.00
0.00
DN
1.00
DV
1.00
BN
1.00
BV
1.00
A
1.00
X
1.00
Xopt
1.00
B
1.00
```

Рисунок 6 – результат работы программы

Заключение: программа была переписана с языка pascal на python, сгенерированы новые данные была проверена её работа.